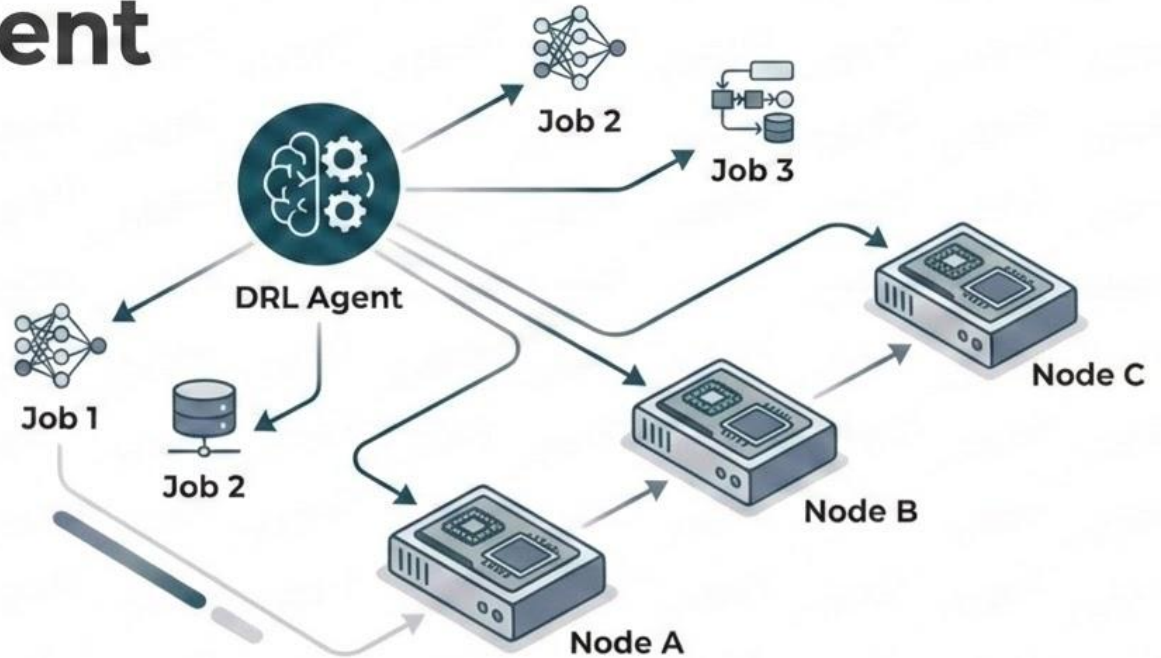


AOT Scheduler for Multi Tenancy System using Deep Reinforcement Learning



The Problem



AllReduce Congestion

Multiple DNN training jobs simultaneously flood core switch links during gradient synchronization, degrading throughput for all tenants.



Reactive RL is Blind

Standard PPO reacts after congestion occurs. It sees a noisy signal because overlap is random — it can't anticipate the deterministic burst schedule.



Wasted Bandwidth

Total demand (25 Gbps across 5 jobs) far exceeds single-link capacity (10 Gbps). Without scheduling, jobs compete instead of cooperating.

System Architecture

1. AOT PROFILER

Observe & Learn

- Monitors first 300 ticks
- Learns burst rhythm per job
- Builds timing model:
 $\text{active}(t) = ((t - \text{off}) \% \text{period}) \geq T_c$

2. EDF Choreographer

Plan & Optimise

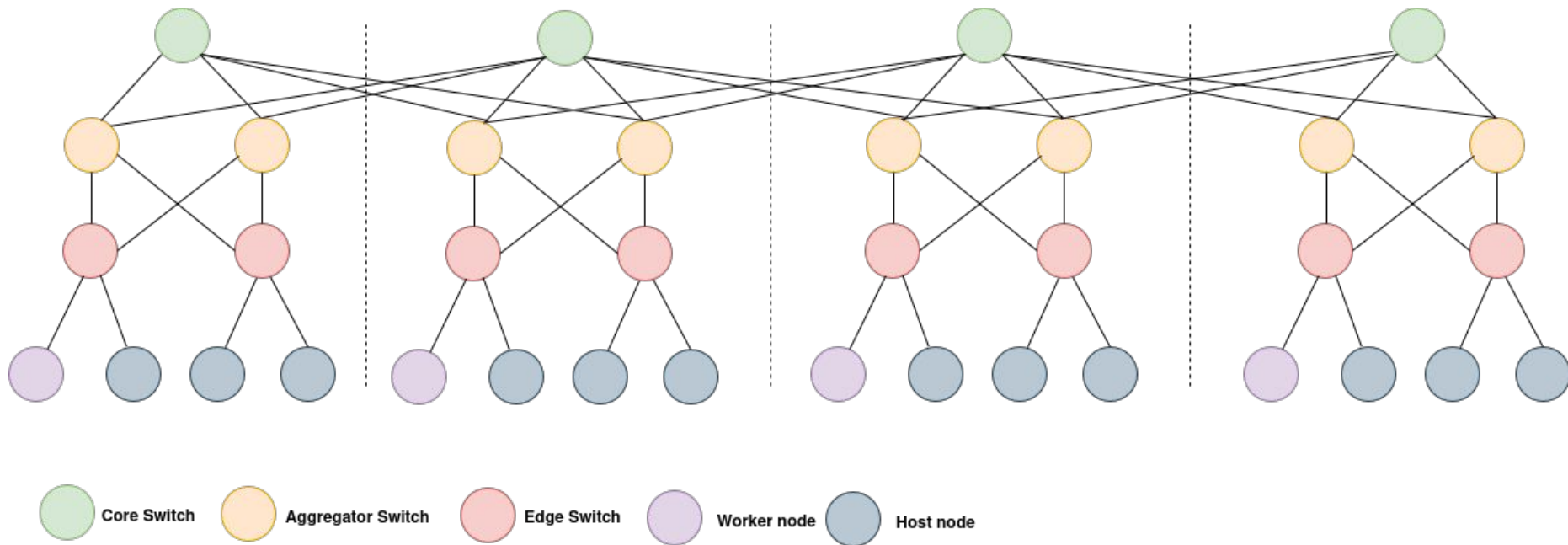
- Fixes Job 0 (highest priority)
- Sweeps 3,000 random offsets
- Minimises $\sum \text{active}^2$ for ticks with ≥ 2 jobs
- Achieves 20 fewer overlap ticks (-7%)

3. PPO Agent

Learn & Act

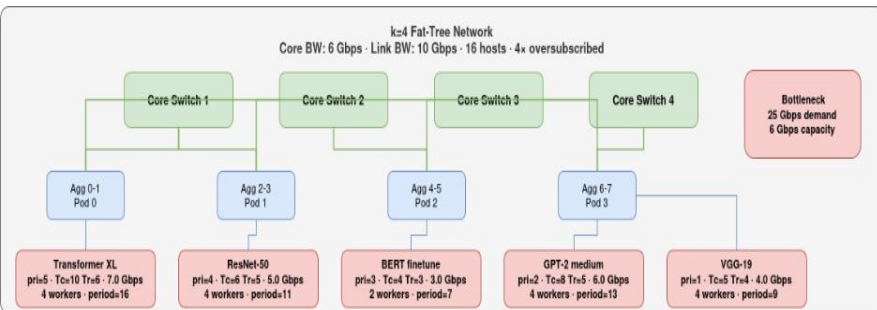
- Runs on choreographed schedule
- Cleaner congestion signal
- Chooses Ring / Tree per job per tick
- Faster convergence, higher reward

Fat-Tree Topology — $k=4$, Cross-Pod Placement

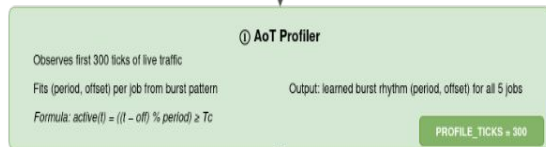


Cross-pod placement: every AllReduce traverses core switches → real congestion | Total demand: 25 Gbps vs core BW: 6 Gbps

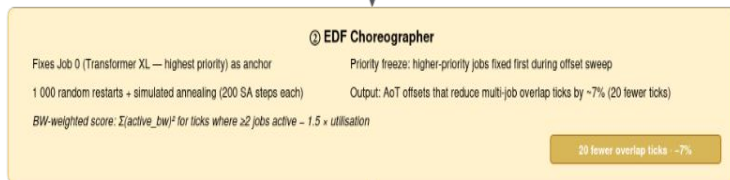
Architecture



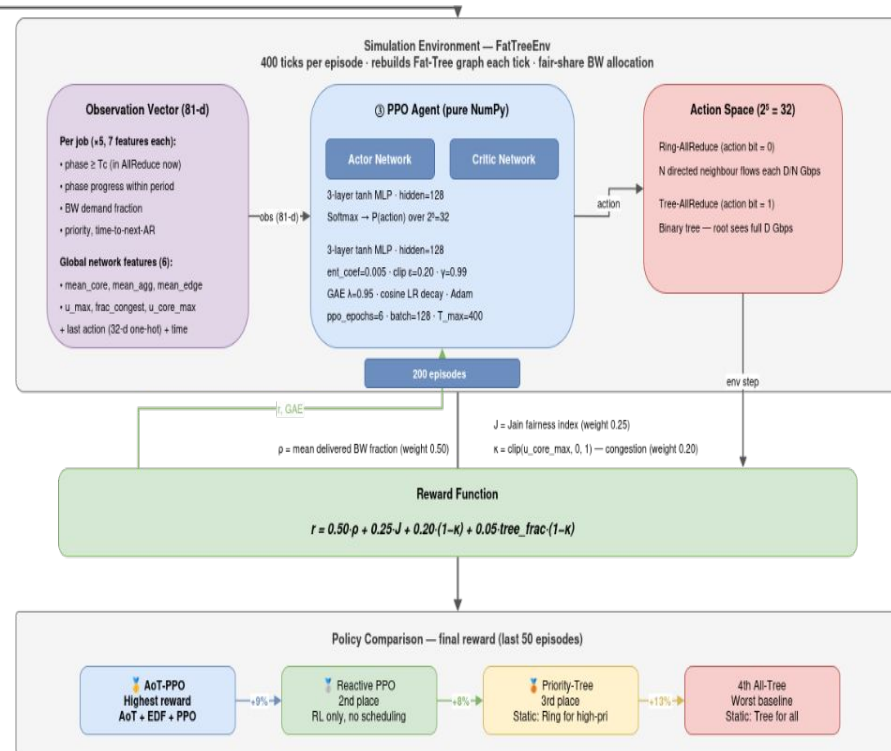
traffic observations (300 ticks)



learned (period, offset) per job



choreographed AoT offsets



Five-Tenant Workload

Job	Model	Workers	Tc (compute)	Tr (AllReduce)	BW (Gbps)	Priority	Period
0	Transformer XL	4	10	6	7.0	5 (highest)	16
1	ResNet-50	4	6	5	5.0	4	11
2	BERT finetune	2	4	3	3.0	3	7
3	GPT-2 medium	4	8	5	6.0	2	13
4	VGG-19	4	5	4	4.0	1 (lowest)	9

Reward Function

$$r = 0.50 \cdot p + 0.25 \cdot J + 0.20 \cdot (1 - K) + 0.05 \cdot \text{tree_frac} \cdot (1 - K)$$



weight = 0.50

Mean delivered BW fraction across active jobs (from fair-share model). Primary throughput objective.



weight = 0.25

Jain fairness index. Rewards equitable BW distribution across all tenants.



weight = 0.20

Core utilisation: $\min(u_core_max, 1.0)$. Clipped to $[0, 1]$ to prevent $\lambda < 0$ bug. Penalises congestion.



weight = 0.05

$\text{tree_frac} \times (1 - K)$. Small structural bias — penalises Tree AllReduce when network is already loaded.

PPO Agent

ent_coef

0.005

Low entropy — prevents divergence (was 0.02 in v2, caused divergence after ep 100)

LR schedule

Cosine decay

Learning rate decays with cosine annealing over training

clip_ratio

0.2

Standard PPO clipping for stable policy updates

gamma

0.99

High discount — values long-term congestion reduction

n_steps

200

Rollout length per episode

Episodes

200

Total training episodes; assertions verified at ep 200

State space: [tick, active jobs, u_max, u_core, rho, delivery per job] | Action: collective type per job (Ring=0, Tree=1)

Results

```
=====
FINAL RESULTS — AoT-PP0
=====
```

Policy	Reward		ρ	Jain	κ	λ
All-Ring	81.29±0.00	0.119	0.566	0.932	+0.0000	
All-Tree	66.58±0.00	0.101	0.391	0.926	+1.0000	
Priority-Tree	80.84±0.00	0.134	0.491	0.926	+0.6000	
Reactive PP0	60.20±1.11	0.080	0.394	0.932	+0.6165	
AoT-PP0 (ours)	86.32±1.63	0.142	0.540	0.931	+0.5204	

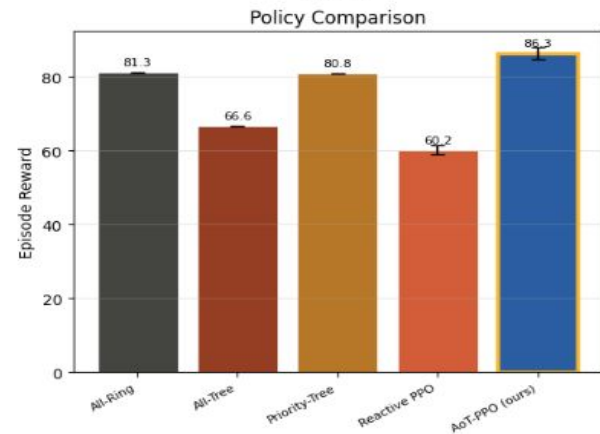
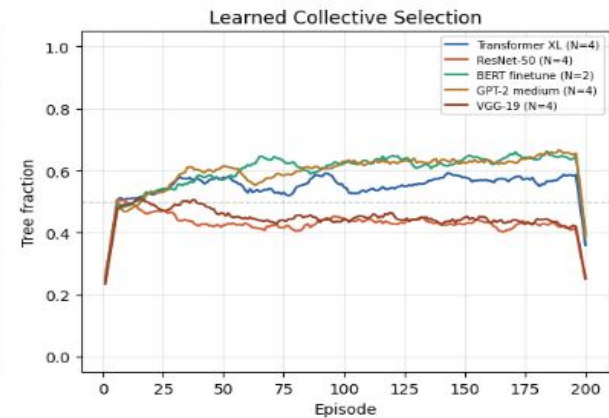
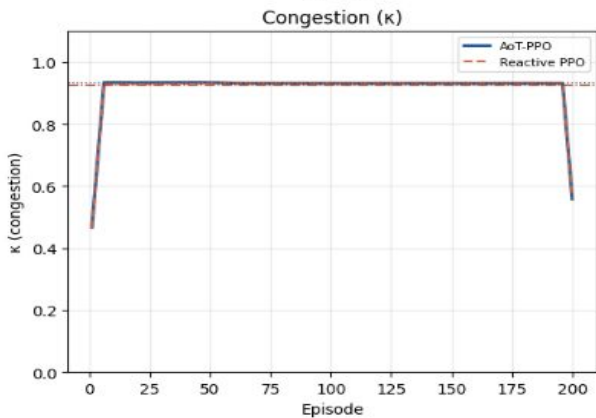
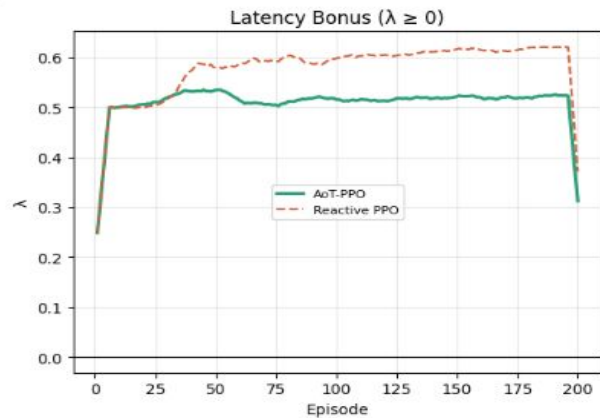
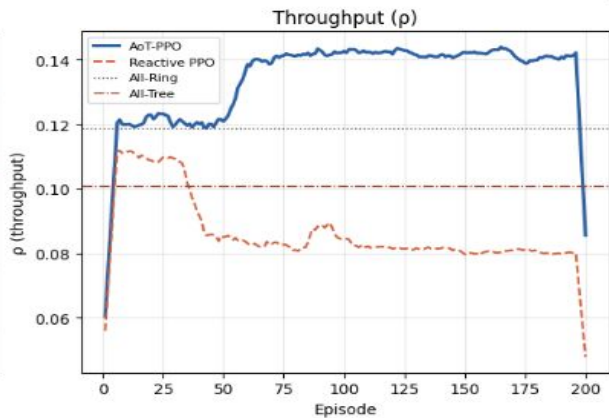
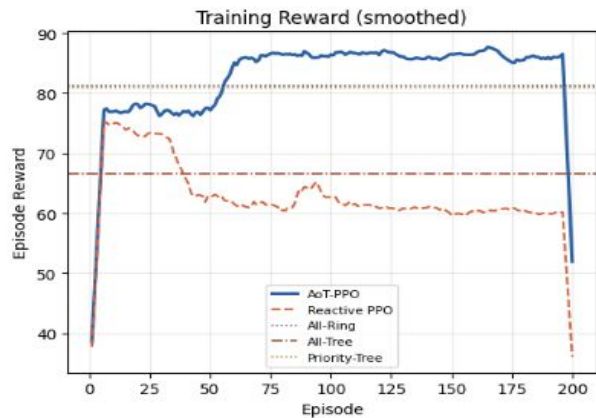
```
=====
```

AoT-PP0 improvements:

vs All-Ring : +5.04 pts
vs All-Tree : +19.74 pts
vs Priority-Tree : +5.48 pts
vs Reactive PP0 : +26.13 pts

Results

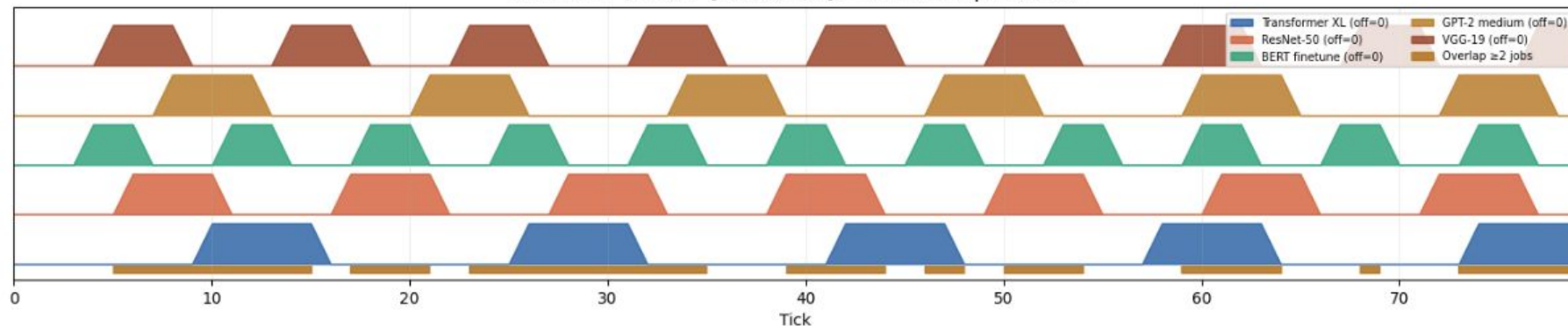
AoT-PPO vs Reactive PPO — Fat-Tree Collective Scheduling



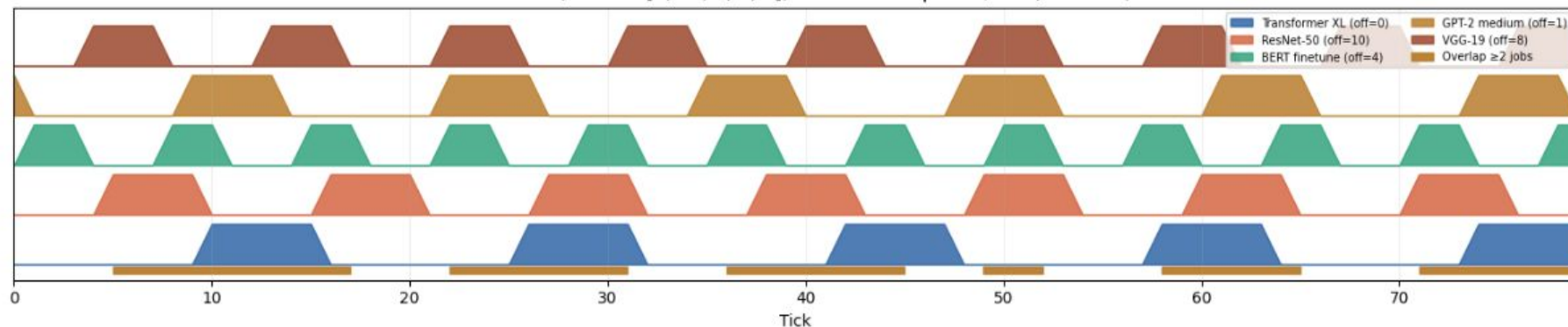
Results..Cont'd

AoT Choreographer: Burst Rhythm Before vs After Offset Optimisation
(amber bars = ≥ 2 jobs overlapping)

Before AoT (offsets=[0, 0, 0, 0, 0]) — 286 overlap ticks/400



After AoT (offsets=[0, 10, 4, 1, 8]) — 275 overlap ticks/400 (11 fewer)



Summary & Conclusions



AoT Profiling Works

300-tick window is sufficient to learn AllReduce burst rhythms.
Deterministic ML traffic enables reliable predictive scheduling.



EDF Scheduling is Effective

3,000 random offset trials reduce multi-job overlap ticks by ~7% (20 ticks). Simple brute-force beats reactive adjustment.



PPO Benefits from Cleaner Signal

Choreographed schedule removes random overlap noise → faster convergence, higher final reward than Reactive PPO.



Physics-Aware Reward Matters

Clipping $\kappa \in [0,1]$ and tuning `ent_coef=0.005 + cosine LR` were critical for stable training and correct reward direction.

Our work addresses a core big data problem by optimizing communication among multiple distributed jobs, similar to how systems like Spark manage shuffle traffic across clusters. By reducing network contention through scheduling (AoT) and adaptive policies (PPO), it improves throughput and efficiency in large-scale data processing environments.

THANK YOU